

QuaRot: 회전된 LLM의 W4A4KV4 추론

QuaRot: Outlier-Free 4-Bit Inference in Rotated LLMs

Saleh Ashkboos et al. (2024)

Primary technical text: [arXiv:2404.00456v2](https://arxiv.org/abs/2404.00456v2) · orthogonal basis change 뒤
weight·activation·KV cache를 4-bit로 양자화한다

문제에서 해법까지의 한 줄

Coordinate outlier가 shared low-bit range를 지배하는 문제를 representation change로 완화한다

Stage	Problem or action	Boundary
Outlier	몇 coordinate만 매우 큼	4-bit scale을 소비
Rotation	Randomized Hadamard로 혼합	값별 rescaling 아님
Compensation	인접 weight도 함께 변환	FP function 보존
Quantization	W4A4KV4 적용	Clipping·rounding error 존재
Kernel	INT4 GEMM과 cache 활용	모든 연산이 INT4는 아님

QuaRot은 Hadamard가 GPTQ를 대체하는 방법이 아니라 rotation 뒤 PTQ를 적용하는 방법이다

하나의 큰 coordinate가 4-bit range를 지배한다

교육용 row $[0.2, 4.0, -0.1, 0.1]$ 에서 max magnitude는 4.0이다

01	Activation row	$[0.2, 4.0, -0.1, 0.1]$
↓		
02	Shared scale	Symmetric INT4는 row max로 scale 결정
↓		
03	Range usage	4.0이 positive endpoint를 차지
↓		
04	Small coordinates	0 근처 값은 적은 quantization levels 사용
↓		
05	Error pressure	Rounding으로 상대 오차가 커질 수 있음

Outlier-free는 rotated distribution을 뜻하며 quantization error는 남는다

2차원 normalized Hadamard를 직접 계산한다

교육용 예시이며 reported experiment가 아니다: $H = (1/\sqrt{2}) \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$

Object	Row 1	Row 2	Check
H	$\{1, 1\} / \sqrt{2}$	$\{1, -1\} / \sqrt{2}$	Orthogonal rows
$H^T H$	$\{1, 0\}$	$\{0, 1\}$	Identity I
x	$\{4, 0\}$	$xH = \{2\sqrt{2}, 2\sqrt{2}\}$	Max 4 $\rightarrow$ 2.828
w	$\{3, 1\}^T$	$H^T w = \{2\sqrt{2}, \sqrt{2}\}^T$	Compensated

$H^T H = I$ 이므로 activation과 weight를 함께 바꾸면 output이 보존된다

회전 전후 dot product는 12로 같다

x와 w 양쪽을 일관되게 변환해야 equality가 성립한다

Path	Activation	Weight	Output
Original	$x = [4, 0]$	$w = [3, 1]^T$	$xw = 12$
Rotated	$xH = [2\sqrt{2}, 2\sqrt{2}]$	$H^T w = [2\sqrt{2}, \sqrt{2}]^T$	$8 + 4 = 12$
Identity used	xH	$H^T w$	$xHH^T w = xw$

회전 보존성은 quantization 전 full-precision algebra에 대한 statement다

Quantization error는 별도의 단계에서 생긴다

교육용 unclipped symmetric INT4 예시: $qmax=7$, $scale=\max(\text{abs}(z))/7=4/7$

Item	Value	Result
FP row z	$[4, 0.4, -0.2, 0.1]$	Max magnitude 4
Integer code	$\text{round}(z / 0.5714)$	$[7, 1, 0, 0]$
Dequantized	$\text{code} \times 0.5714$	$[4, 0.5714, 0, 0]$
Error	$\text{dequantized} - z$	$[0, 0.1714, 0.2, -0.1]$

Orthogonal equality와 low-bit error는 서로 다른 단계다

Transformer에서 rotation을 합법화하는 조건

RMSNorm scale을 adjacent weights에 흡수한 norm-only form에서 commutation을 사용한다

Condition	Operation	Meaning
Orthogonal Q	$QQ^T=I$	Inverse cancellation
RMSNorm scale	alpha를 weights에 흡수	Norm-only relation 사용
Commutation	$\text{RMSNorm}(X)=\text{RMSNorm}(XQ^T)Q$	Residual basis 이동
Exact scope	Before clipping and quantization	FP function 보존
Implementation	Factors를 weights에 fuse	Runtime matmul 제거 가능

Eq. 3은 arbitrary normalization이나 finite-precision pipeline 전체의 equality가 아니다

Global residual-stream rotation은 weight에 bake한다

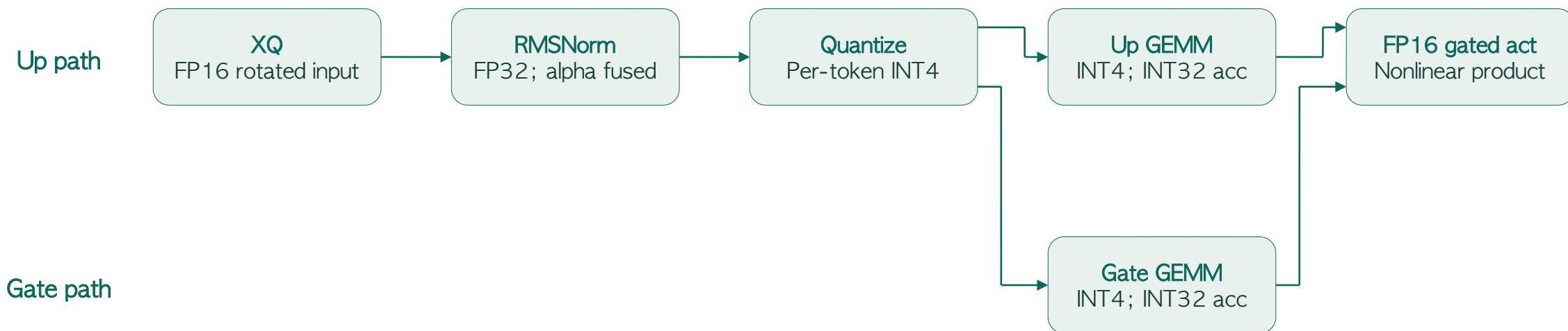
$Q=H \text{ diag}(s)$ 를 고르고 block 경계의 representation을 XQ basis로 바꾼다

01	Choose Q	Randomized Hadamard H diag(s)
↓		
02	Residual basis	X를 XQ representation으로 해석
↓		
03	Input weights	$Q^T \text{diag}(\alpha) W$ 형태로 fuse
↓		
04	Output weights	Output-side matrix에 Q를 fuse
↓		
05	Runtime result	별도 global Q matmul 없이 rotated state 유지

Fusion은 별도 runtime matmul을 없애지만 rotation의 역할은 남는다

FFN 경로 1 : gate product까지

첫 장의 출력 상태 FP16 gated activation이 다음 장의 입력 상태와 정확히 같다



Gating nonlinearity 때문에 FFN 내부 rotation 전체를 neighboring weights로 넘길 수 없다

FFN 경로 2: online H와 down projection

FP16 gated activation에서 시작해 online full-precision Hadamard를 실제 runtime에 수행한다



Stored down matrix는 HWdownQ이고 그 앞의 activation Hadamard는 runtime cost를 가진다

Fused rotation과 online rotation을 구분한다

수식에 H나 Q가 보인다고 모두 별도 runtime matrix multiplication인 것은 아니다

Location	Treatment	Runtime transform
Residual stream	Factors fused into weights	No separate global matmul
FFN internal	H fused into Wdown	Activation H remains online
Attention V/out	Head transforms paired	Head mixing may be online
Post-RoPE Q,K	Same head-wise H	Both run online
KV cache	Rotated K,V quantized	Quant/dequant remains

Speed benefit은 fusion과 custom kernel의 조합에서 나온다

Attention value와 output projection의 paired rotation

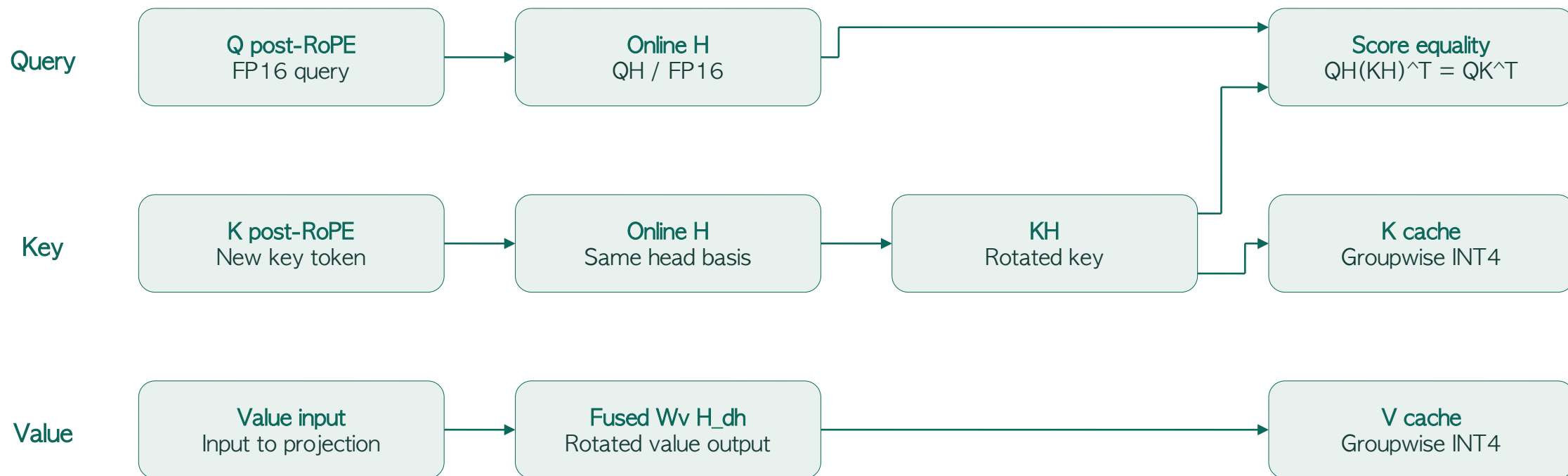
각 head의 value basis change를 output projection의 inverse-side factor가 보상한다

01	Value projection	Per-head W_v is post-multiplied by H_{dh}
↓		
02	Rotated values	Head value coordinates가 mixed basis로 이동
↓		
03	Attention blend	같은 attention weights로 values를 합산
↓		
04	Output projection	Per-head W_{out} is pre-multiplied by H_{dh}
↓		
05	Preserved output	Paired orthogonal factors가 cancel

V만 회전하고 output projection을 그대로 두면 원래 attention output의 보존 설명이 깨진다

Attention의 Q·K·V rotation과 KV cache

Q·K의 H는 RoPE 뒤 online 연산이고, V의 head H는 projection weight에 합쳐진다



KV cache는 rotated K와 V를 4-bit로 저장한다

Post-RoPE caching을 택해 decode step마다 새 token의 Q와 K만 회전한다

01	New token	Post-RoPE Q,K에 head-wise H 적용
↓		
02	Cache write	Rotated K,V를 groupwise INT4 저장
↓		
03	Decode query	Q는 FP16으로 유지
↓		
04	Cache read	K,V segments를 FP16으로 dequantize
↓		
05	Attention	FP16 dot products와 online softmax 수행

KV4는 cache format이며 attention score path 전체가 INT4는 아니다

W4A4KV4의 실제 precision contract

4-bit storage와 operands 사이에도 normalization, accumulation, rescale이 더 높은 precision으로 남는다

01 RMSNorm FP32



02 Linear operands INT4 activations and weights



03 GEMM accumulate INT32



04 Recovered signal Scale 적용 뒤 FP16; online H는 default FP32



05 KV storage INT4; Q와 dequantized cache dot은 FP16

Outlier channel을 남기지 않아도 FP16·FP32 연산은 남는다

Weight·activation·KV는 서로 다른 quantizer를 쓴다

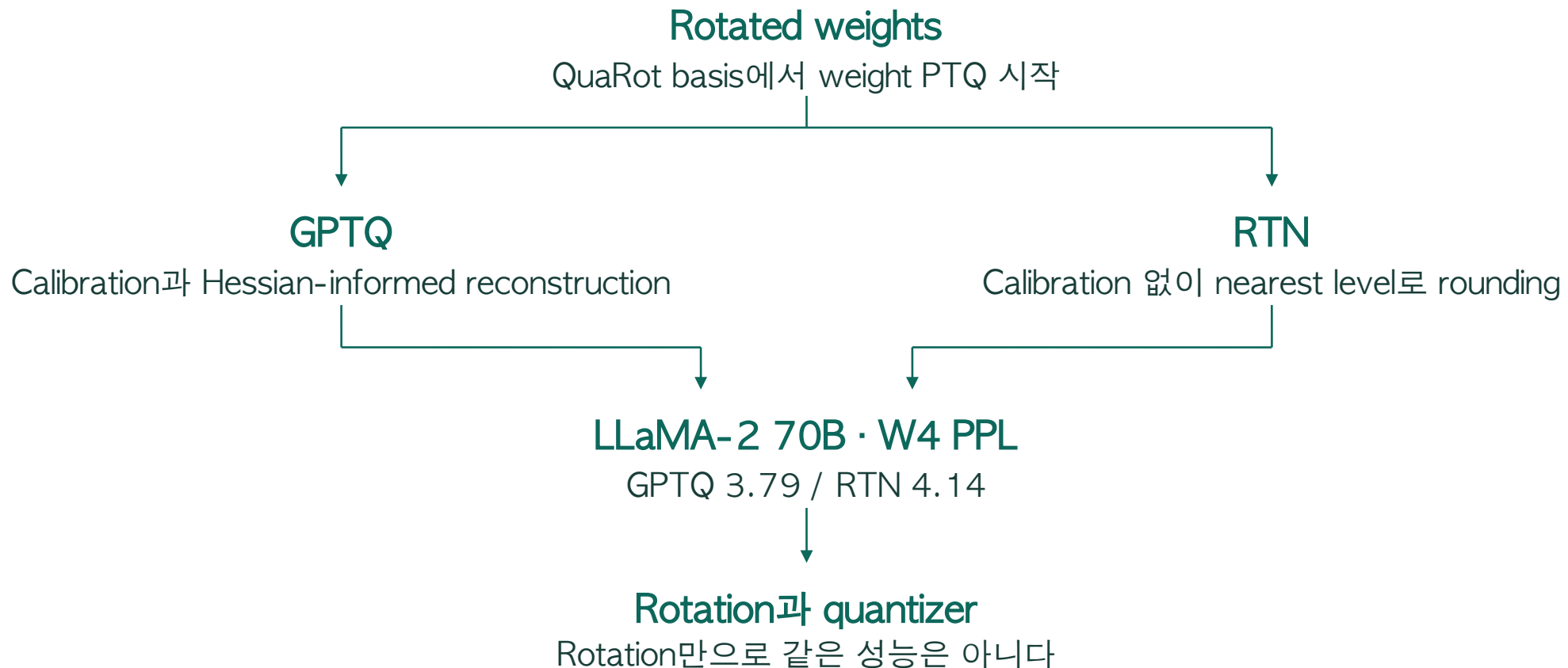
One global 4-bit scale이라는 설명은 실제 implementation을 지운다

Target	Method	Scale or clipping	Default role
Weights	Symmetric per-column GPTQ	Clip searched by squared error	Main W4 result
Activations	Symmetric per-token row	$\max(\text{abs}(\text{row}))/7$; clip 0.9	Online linear input
KV cache	Asymmetric groupwise	group 128; clip 0.95	K,V storage
Weights RTN	Round-to-nearest	No calibration	Ablation baseline

Rotation 뒤에도 default weight PTQ인 GPTQ가 필요하다

Rotation 뒤에도 GPTQ와 RTN은 같은 결과가 아니다

둘은 같은 rotated weights에서 출발하지만 reconstruction 방법과 calibration이 다르다



Calibration과 implementation 조건

Reported accuracy와 speed는 이 setup 안에서 해석한다

Item	Reported setting	Scope
GPTQ calibration	128 WikiText-2 samples $\times$ 2048 tokens	RTN에는 불필요
70B modification	5 min rotation + 2 h GPTQ	One A100
Performance GPU	RTX3090	CUDA 12.1
Kernels	CUTLASS and FlashInfer	Paper implementation
Public code	Authors' QuaRot repository	Reproduction resource

Quantization cost와 inference kernel benchmark는 서로 다른 hardware 조건을 사용했다

Main W4A4KV4 WikiText-2 결과

arXiv v2 Table 1의 LLaMA-2 GPTQ values이며 metric은 perplexity라 낮을수록 좋다

Model	FP16 PPL	QuaRot PPL	Absolute increase
LLaMA-2 7B	5.47	6.10	+0.63
LLaMA-2 13B	4.88	5.40	+0.52
LLaMA-2 70B	3.32	3.79	+0.47

0.47은 70B claim이며 표의 최대 PPL 증가는 7B의 0.63이다

Zero-shot average도 model size별로 다르다

여섯 tasks average의 absolute point change를 보고 rounded retention slogan과 구분한다

Model	FP16 avg	W4A4KV4 avg	Change
LLaMA-2 7B	69.82	65.64	-4.18 points
LLaMA-2 70B	77.07	75.98	-1.09 points
70B retained ratio	100% baseline	about 98.6%	Rounded near 99%

99%는 70B의 rounded ratio이며 7B의 absolute loss를 설명하지 않는다

Prefill speedup의 범위

RTX3090, sequence length 2048의 single transformer block benchmark다

Model	Reported speedup range	Unit	Not claimed
LLaMA-2 7B	1.97-2.16	×	Whole-model latency
LLaMA-2 70B	up to 3.33	×	All batch sizes
Benchmark object	One transformer block	scope	End-to-end service
Hardware	RTX3090	GPU	Hardware-independent

3.33 × 를 전체 model end-to-end latency speedup으로 다시 이름 붙이면 안 된다

Decoding에서 확인된 memory saving

Fig. 4는 batch 16, cache length 256-4096에서 50 tokens decoding을 측정한다

Metric	Reported range	Condition	Interpretation
Peak memory saving	3.63-3.89×	Cache 256-4096	KV cache compression
Stored K,V	INT4	Group size 128	Capacity benefit
Query and dot	FP16	Dequantized cache	Not pure integer
Measurement	Batch 16; 50 tokens	Single block RTX3090	Not full serving stack

KV4 memory 절감 뒤에도 attention용 dequantization은 남는다

작은 decode batch에서는 더 느릴 수도 있다

Quantize와 dequantize overhead가 arithmetic saving보다 클 수 있는 구간이 있다

Regime	Benefit	Overhead	Observed boundary
Large cache pressure	INT4 memory traffic 감소	Quant/dequant 추가	Memory saving 유지
Low batch decode	작은 GEMM workload	Conversion cost 비중 증가	FP16보다 느릴 수 있음
Conclusion	Kernel-dependent	Batch-dependent	Universal speedup 아님

Compression ratio와 latency speedup은 같은 metric이 아니다

같은 source가 보여 주는 LLaMA-3 경계

Ungrouped W4 result는 newer model family에서 훨씬 큰 degradation을 보인다

Model	FP16 PPL	QuaRot W4 PPL	Increase
LLaMA-3 8B	6.14	8.16	+2.02
LLaMA-3 70B	2.86	6.66	+3.80
Paper reading	More sensitive	Ungrouped setting	Scope matters

LLaMA-2 결과만으로 outlier-free makes W4 lossless를 일반화할 수 없다

아무 orthogonal rotation이나 같은 것은 아니다

LLaMA-2 7B W4A4KV4 perplexity에서 randomized Hadamard가 더 좋았다

Rotation	PPL	Comparison	Boundary
Randomized Hadamard	6.10	Main choice	Structured and fast
Random orthogonal	7.45	Worse in Table 8	Still orthogonal
No universal claim	—	Dimension and choice matter	Empirical evidence

Orthogonality만으로 quantized accuracy와 runtime이 같아지지 않는다

QuaRot이 quantization landscape에서 차지하는 위치

핵심 action과 retained precision target을 구분하면 method를 같은 것으로 묶지 않게 된다

Family	Core action	QuaRot과의 차이
GPTQ	Weight reconstruction PTQ	QuaRot 뒤 default weight quantizer
OWQ or AWQ style	Selected channels 유지·보호	QuaRot은 rotated W,A,KV target
SmoothQuant	Feature scaling for W8A8	Orthogonal W4A4KV4가 아님
QulP family	Weight incoherence 중심	QuaRot은 activation과 KV도 처리
KV-only	Cache K,V compression	QuaRot은 W,A,KV를 함께 조정

QuaRot은 representation change와 여러 PTQ components를 결합한 end-to-end scheme이다

QuaRot의 기여와 한계

Coordinate basis를 바꾸어 4-bit quantizer가 다루기 쉬운 representation을 만든다

Claim	Supported reading	Not supported
Invariance	Compensated FP rotation은 function 보존	Quantized bit-exact equality
Outlier-free	Rotated coordinate concentration 완화	Zero quantization error
W4A4KV4	Weights, linear inputs, cache는 INT4	Every operation INT4
Performance	Custom-kernel block benchmark 개선	Universal end-to-end speedup
Accuracy	Strong LLaMA-2 evidence	All model families lossless

Rotation은 PTQ를 돕지만 quantizer·kernel·model 조건은 남는다